AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Wiktor Matusik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Przykład 4

1.2. Instalacja 6

2. Analiza problemu **8**

2.1. Zastosowanie algorytmu / struktury danych 8

2.2. Sposób działania operacji dodawania 8

2.3. Przykład działania (Krok po kroku) 9

2.4. Opis wykorzystanych narzędzi 9

3. Projektowanie **10**

3.1. Diagram klas i struktura plików 10

3.2. Diagram przepływu sterowania dla dodawania elementu 10

4. Implementacja **12**

4.1. Klasa Wezel 12

4.2. Wzorzec Fabryki 12

4.3. Klasa ListaDwukierunkowa 13

4.4. Funkcja główna programu 16

5. Wnioski **17**

Literatura **18**

Spis rysunków **19**

Spis tabel **20**

Spis listingów **21**

1. Ogólne określenie wymagań

Celem projektu jest zaimplementowanie dynamicznej struktury danych – listy dwukierunkowej – działającej w pamięci dynamicznej (stercie) przy użyciu języka C++.

Aplikacja oraz proces jej tworzenia muszą spełniać następujące wymagania funkcjonalne i organizacyjne:

- Wykorzystanie wzorca projektowego Fabryka (*Factory*) oraz Iteratora przy pisaniu implementacji.
- Podział projektu na moduły – każda klasa musi znajdować się w osobnym pliku.
- Implementacja następujących metod i funkcjonalności klasy:
  - Dodaj element na początek listy.
  - Dodaj element na koniec listy.
  - Dodaj element pod wskazany indeks.
  - Usuń element z początku listy.
  - Usuń element z końca listy.
  - Usuń element spod wskazanego indeksu.
  - Wyświetl całą listę.
  - Wyświetl listę w odwrotnej kolejności.
  - Wyświetl następny element.
  - Wyświetl poprzedni element.
  - Czyść całą listę.
- Działanie zaimplementowanych klas i metod należy przetestować w funkcji `main`.
- Podczas wykonywania projektu należy zrealizować następujące wytyczne technologiczne i metodyczne:
  - Wygenerować dokumentację automatycznie za pomocą programu *Doxygen* do pliku LaTeX z rozszerzeniem `.pdf`.
  - Do projektu dołączyć dokumentację projektową w języku LaTeX (zgodnie z szablonem udostępnionym przez prowadzącego).

  - Stworzyć konto na platformie GitHub (osoby posiadające konto mogą działać na swoim koncie).
  - Można zainstalować wtyczkę do Visual Studio Code lub Visual Studio (zależy od środowiska, z którego będziecie korzystać).
  - Używać systemu Git oraz serwisu GitHub przy pisaniu programu (można wypróbować wtyczki przy commitowaniu).

- Przy pisaniu listy dwukierunkowej należy przetestować następujące scenariusze z zastosowaniem systemu Git:
  - Wykonanie co najmniej 5 commitów w historii repozytorium.
  - Co najmniej jedno cofnięcie się w historii o dwa commity wstecz.
  - Usunięcie jednego wybranego commita.
  - Pobranie projektu z GitHuba do innej lokalizacji na komputerze, dopisanie kolejnej metody w pobranym projekcie, zapisaną zmianę wysłać na serwer. Następnie pobranie zmian z GitHuba do pierwszej lokalizacji, dopisanie nowej metody i zapisanie zmiany na serwer (obie lokalizacje mogą znajdować się na jednym komputerze w różnych katalogach lub na dwóch komputerach).
  - Celowe usunięcie jakiegoś pliku w katalogu projektu (w taki sposób, aby program przestał się kompilować), a następnie pobranie ostatniej poprawnej wersji z GitHuba, aby znów projekt się kompilował.

1.1. Przykład

Tak zaczynamy pisanie pierwszego akapitu. Jeśli chcemy napisać przypis do bibliografii wykonujemy to w ten sposób[1].



Rys. 1.1. Logo

Tutaj może coś być wpisane.
Tutaj może coś być wpisane[2]. Rysunek 1.1 (s. 4) pokazuje przykładową ilustrację.

[1] Przykład odnośnika do książki[1].
[2] Przykład odnośnika do strony www[2].

Tab. 1.1. Tabelka przykładowa

U_n	I_{zw}
kV	%
7.2	100

Listing kodu

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

/*
liczby pseldolosowe
*/

int main(int argc, char** argv) {

  int tab[10][10];

  for(int i=0;i<10;i++)
  for(int j=0;j<10;j++)
  tab[i][j]=0;

  srand(time(NULL));    //generowanie z czasu
  int min=3;
  int max=7;
  for(int i=0;i<10;i++)
  for(int j=0;j<10;j++)
  tab[i][j]=(rand()%(max-min+1))+min;

  for(int i=0;i<10;i++)
  {
    for(int j=0;j<10;j++)
    cout<<tab[i][j]<<" ";
    cout<<endl;
  }

  return 0;
}
```

Listing 1. Przykładowy kod 001

Tutaj może coś być wpisane. Tutaj może coś być wpisane. Tutaj może coś być

wpisane. Tabela 1.1 (s. 5) pokazuje sposoby użycia trybu matematycznego.

Kod 1 (s. 5) przedstawia sposób generowania liczb pseudolosowych. Kod 2 (s. 6) przedstawia generowanie pliku HTML.

Alternatywna metoda wklejenia kodu:

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char** argv) {

  ofstream plik("strona.html");
  if(!plik)
    cout<<"blad zapisu pliku";
  else
    {
    plik<<"<html>";
    plik<<"<head><title>Moja pierwsza strona www</title></head>";
    plik<<"<body>Strona WWW</body>";
    plik<<"</html>";

    cout<<"Wygenerowana strona";
    }
    plik.close();


    return 0;
}
```

Listing 2. Przykładowy kod 002

1.2. Instalacja

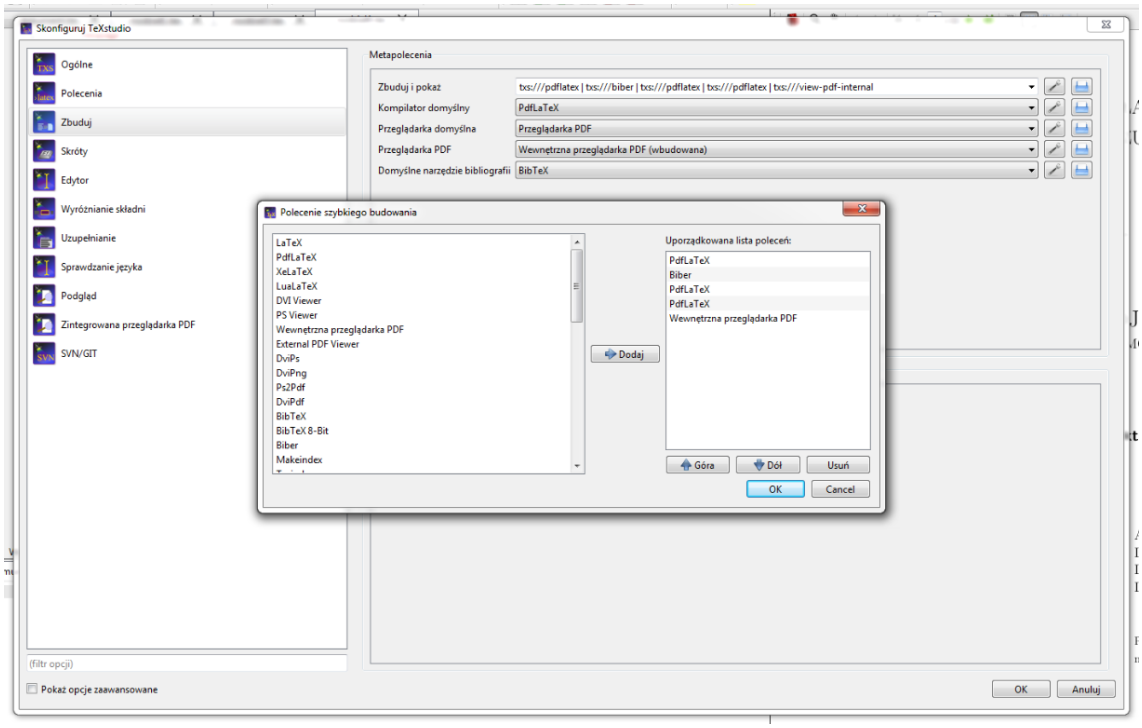
Poniżej są opisane kroki potrzebne do instalacji LaTeX'a oraz do używania tego szablonu.

Na początku instalujemy TeXLive[3]. Ściągamy plik instalacyjny, zajmuje około 25MB. Podczas instalacji można wybrać do zainstalowania różne kolekcje pakietów. Jeśli nie ma problemów z miejscem na dysku to można zainstalować wszystkie, wtedy nie będzie problemu z brakującymi pakietami i błędami. Po wybraniu kolekcji brakujące pliki są pobierane z internetu. Pełna instalacja programu zajmuje około 8GB. Najlepiej zostawić instalację na noc, ponieważ proces zabiera sporo czasu.

[3]Instalka na stronie https://www.tug.org/texlive/acquire-netinstall.html[3].

Warto ustawić komputer tak, aby się nie wyłączył lub nie uśpił. Warto także przed instalacją zablokować antywirusa, ponieważ może blokować niektóre z komponentów.

Następnie instalujemy TEXstudio[4]. Ściągamy plik instalacyjny zajmujący około 120MB. Instalacja przebiega standardowo.



Rys. 1.2. Ustawienie TeXstudio

Następnym krokiem jest ustawienie w TEXStudio kolejności budowania projektu. Należy wybrać zakładkę: „Opcje/Konfiguruj TEXstudio...". W otwartym oknie przechodzimy na zakładkę „Zbuduj". Na rysunku 1.2 (s. 7) pokazany jest zrzut ekranu z konfiguracją. W linijce „Zbuduj i pokaż" klikamy ikonę klucza, żeby przejść do konfiguracji polecenia. W otwartym oknie ustawić kolejność tak jak pokazano na rysunku.

[4]Plik instalacyjny na stronie https://www.texstudio.org[4].

2. Analiza problemu

W procesie projektowania i implementacji listy dwukierunkowej na stercie w języku C++ przeanalizowano zagadnienia tworzenia dynamicznej struktury danych operującej na trzech podstawowych metodach dodawania elementów:

- **Zarządzanie pamięcią dynamiczną (sterta):** Elementy listy są alokowane dynamicznie na stercie za pomocą operatora `new`. Każdy węzeł przechowuje wartość oraz dwa wskaźniki: na element następny (`nastepny`) i poprzedni (`poprzedni`).
- **Zastosowanie wzorca projektowego Fabryka (*Factory*):** W celu oddzielenia procesu alokacji pamięci dla węzłów od samej logiki listy, dodawanie elementów wykorzystuje klasę `FabrykaWezlow`, która tworzy i zwraca nowe obiekty klasy `Wezel`.

2.1. Zastosowanie algorytmu / struktury danych

Lista dwukierunkowa oraz operacja dodawania elementów do niej znajduje zastosowanie w:

- Budowaniu dynamicznych kolejek zadań (dodawanie na koniec lub z priorytetem pod wskazany indeks).
- Tworzeniu historii operacji (np. dodawanie nowych akcji użytkownika na początek lub koniec listy).
- Systemach zarządzania pamięcią buforową.

2.2. Sposób działania operacji dodawania

- **Dodawanie na początek (`dodajNaPoczatek`):** Tworzony jest nowy węzeł, którego wskaźnik `nastepny` wskazuje na dotychczasowy pierwszy węzeł. Jeśli lista była pusta, nowy węzeł staje się jednocześnie początkiem i końcem listy.
- **Dodawanie na koniec (`dodajNaKoniec`):** Nowy węzeł jest dołączany za dotychczasowym końcem listy (`koniec->nastepny`). Jego wskaźnik `poprzedni` wskazuje na stary koniec, po czym wskaźnik `koniec` jest aktualizowany.
- **Dodawanie pod wskazany indeks (`dodajPodIndeks`):** Algorytm przechodzi od początku listy do wskazanego miejsca, a następnie wstawia nowy węzeł poprzez zmianę wskaźników `nastepny` oraz `poprzedni` w sąsiednich węzłach.

2.3. Przykład działania (Krok po kroku)

Przeanalizujmy ręczny przebieg dodawania trzech elementów do pustej listy:

1. **Stan początkowy:** Lista jest pusta. `poczatek = nullptr`, `koniec = nullptr`, `rozmiar = 0`.
2. **Krok 1: Wywołanie** `dodajNaPoczatek(10)`
   - Fabryka tworzy nowy węzeł z wartością 10.
   - Ponieważ lista była pusta, zarówno `poczatek`, jak i `koniec` wskazują na węzeł 10.
   - Stan listy: `10`. Rozmiar = 1.
3. **Krok 2: Wywołanie** `dodajNaKoniec(30)`
   - Fabryka tworzy węzeł z wartością 30.
   - Dotychczasowy koniec (węzeł 10) ustawia `nastepny` na węzeł 30.
   - Węzeł 30 ustawia `poprzedni` na węzeł 10.
   - Wskaźnik `koniec` przesuwa się na węzeł 30.
   - Stan listy: `10` $\leftrightarrow$ `30`. Rozmiar = 2.
4. **Krok 3: Wywołanie** `dodajPodIndeks(1, 20)`
   - Wskaźnik przechodzi do węzła pod indeksem 1 (węzeł z wartością 30).
   - Fabryka tworzy nowy węzeł z wartością 20.
   - Wskaźniki nowego węzła ustawiane są na: `poprzedni = 10`, `nastepny = 30`.
   - Aktualizacja wskaźników sąsiadów: węzeł 10 wskazuje w prawo na 20, a węzeł 30 wskazuje w lewo na 20.
   - Ostateczny stan listy: `10` $\leftrightarrow$ `20` $\leftrightarrow$ `30`. Rozmiar = 3.

2.4. Opis wykorzystanych narzędzi

- **Język C++ oraz kompilator GCC:** Użyte do alokacji obiektów na stercie za pomocą operatorów dynamicznego zarządzania pamięcią.
- **Git oraz GitHub:** Używane do kontroli wersji projektu i przechowywania plików źródłowych.

3. Projektowanie

W projekcie postawiono na programowanie obiektowe i podział kodu na mniejsze części. Rozdzieliliśmy zadania tak, żeby każda klasa miała swoje pliki, co sprawia, że jest on bardziej przejrzysty.

3.1. Diagram klas i struktura plików

Projekt składa się z następujących modułów:

- **Klasa** `Wezel` **(**`Wezel.h`**,** `Wezel.cpp`**):** Odpowiada za pojedynczy element listy przechowywany na stercie. Zawiera pole wartości całkowitoliczbowej (`wartosc`) oraz wskaźniki na węzeł następny (`nastepny`) i poprzedni (`poprzedni`).
- **Wzorzec Fabryki (**`FabrykaWezlow.h`**,** `FabrykaWezlow.cpp`**):**
  - Klasa abstrakcyjna `FabrykaWezlow` definiuje czysto wirtualną metodę tworzenia węzłów.
  - Klasa pochodna `BetonowaFabrykaWezlow` implementuje tę metodę, wykonując rzeczywistą alokację pamięci dynamicznej za pomocą operatora `new`.
- **Klasa** `ListaDwukierunkowa` **(**`ListaDwukierunkowa.h`**,** `ListaDwukierunkowa.cpp`**):** Zarządza całą strukturą danych, przechowując wskaźniki na początek (`poczatek`) i koniec (`koniec`) listy oraz aktualny `rozmiar`. Implementuje metody:
  - `dodajNaPoczatek(int wartosc)`
  - `dodajNaKoniec(int wartosc)`
  - `dodajPodIndeks(int indeks, int wartosc)`

3.2. Diagram przepływu sterowania dla dodawania elementu

Proces dodawania elementu pod wskazany indeks przebiega według następującego algorytmu:

1. Sprawdzenie poprawności podanego indeksu (czy $0 \leq \text{indeks} \leq \text{rozmiar}$). W przypadku błędu zgłaszany jest wyjątek.
2. Jeśli indeks wynosi 0, wywoływana jest bezpośrednio metoda `dodajNaPoczatek`.
3. Jeśli indeks jest równy aktualnemu rozmiarowi, wywoływana jest metoda `dodajNaKoniec`.

4. W pozostałych przypadkach następuje iteracyjne przejście do węzła znajdującego się pod zadanym indeksem, utworzenie nowego węzła przy użyciu fabryki oraz odpowiednie przepięcie wskaźników `nastepny` i `poprzedni` sąsiadujących elementów.

4. Implementacja

W tej sekcji przedstawiono kompletny kod źródłowy programu, podzielony na osobne pliki nagłówkowe i implementacyjne zgodnie z wymaganiami projektowymi.

4.1. Klasa Wezel

Pliki `Wezel.h` oraz `Wezel.cpp` definiują strukturę pojedynczego elementu listy przechowywanego na stercie.

```
#ifndef WEZEL_H
#define WEZEL_H

class Wezel {
public:
    int wartosc;
    Wezel* nastepny;
    Wezel* poprzedni;

    Wezel(int val, Wezel* pop = nullptr, Wezel* nast = nullptr);
};

#endif
```

Listing 3. Plik nagłówkowy Wezel.h

```
#include "Wezel.h"

Wezel::Wezel(int val, Wezel* pop, Wezel* nast)
    : wartosc(val), poprzedni(pop), nastepny(nast) {}
```

Listing 4. Plik implementacyjny Wezel.cpp

4.2. Wzorzec Fabryki

Pliki `fabryka.h` oraz `fabryka.cpp` realizują wzorzec projektowy Factory, odpowiedzialny za alokację pamięci dla nowych węzłów.

```
#ifndef FABRYKA_H
#define FABRYKA_H

#include "Wezel.h"

class FabrykaWezlow {
public:
```

```
8       virtual ~FabrykaWezlow() = default;
9       virtual Wezel* utworzWezel(int wartosc, Wezel* poprzedni =
        nullptr, Wezel* nastepny = nullptr) = 0;
10  };
11
12  class FabrykaWezlowImpl : public FabrykaWezlow {
13  public:
14      Wezel* utworzWezel(int wartosc, Wezel* poprzedni = nullptr,
        Wezel* nastepny = nullptr) override;
15  };
16
17  #endif
```

Listing 5. Plik nagłówkowy fabryka.h

```
1   #include "fabryka.h"
2
3   Wezel* FabrykaWezlowImpl::utworzWezel(int wartosc, Wezel* poprzedni
        , Wezel* nastepny) {
4       return new Wezel(wartosc, poprzedni, nastepny);
5   }
```

Listing 6. Plik implementacyjny fabryka.cpp

4.3. Klasa ListaDwukierunkowa

Pliki `ListaDwukierunkowa.h` oraz `ListaDwukierunkowa.cpp` zawierają logikę zarządzania strukturą oraz metody dodawania elementów.

```
1   #ifndef LISTA_DWUKIERUNKOWA_H
2   #define LISTA_DWUKIERUNKOWA_H
3
4   #include "Wezel.h"
5   #include "fabryka.h"
6
7   class ListaDwukierunkowa {
8   private:
9       Wezel* poczatek;
10      Wezel* koniec;
11      int rozmiar;
12      FabrykaWezlow* fabryka;
13
14  public:
15      ListaDwukierunkowa(FabrykaWezlow* fabryka);
16      ~ListaDwukierunkowa();
```

```
17
18      void dodajNaPoczatek(int wartosc);
19      void dodajNaKoniec(int wartosc);
20      void dodajPodIndeks(int indeks, int wartosc);
21
22      int pobierzRozmiar() const;
23      void wyswietl() const;
24  };
25
26  #endif
```

Listing 7. Plik nagłówkowy ListaDwukierunkowa.h

```
1   #include "ListaDwukierunkowa.h"
2   #include <iostream>
3   #include <stdexcept>
4
5   ListaDwukierunkowa::ListaDwukierunkowa(FabrykaWezlow* fab)
6       : poczatek(nullptr), koniec(nullptr), rozmiar(0), fabryka(fab)
      {}
7
8   ListaDwukierunkowa::~ListaDwukierunkowa() {
9       Wezel* temp = poczatek;
10      while (temp != nullptr) {
11          Wezel* nastepny = temp->nastepny;
12          delete temp;
13          temp = nastepny;
14      }
15  }
16
17  void ListaDwukierunkowa::dodajNaPoczatek(int wartosc) {
18      Wezel* nowyWezel = fabryka->utworzWezel(wartosc, nullptr,
      poczatek);
19      if (poczatek == nullptr) {
20          koniec = nowyWezel;
21      } else {
22          poczatek->poprzedni = nowyWezel;
23      }
24      poczatek = nowyWezel;
25      rozmiar++;
26  }
27
28  void ListaDwukierunkowa::dodajNaKoniec(int wartosc) {
29      Wezel* nowyWezel = fabryka->utworzWezel(wartosc, koniec,
      nullptr);
30      if (koniec == nullptr) {
```

```
31          poczatek = nowyWezel;
32      } else {
33          koniec->nastepny = nowyWezel;
34      }
35      koniec = nowyWezel;
36      rozmiar++;
37  }
38
39  void ListaDwukierunkowa::dodajPodIndeks(int indeks, int wartosc) {
40      if (indeks < 0 || indeks > rozmiar) {
41          throw std::out_of_range("Indeks poza zakresem listy!");
42      }
43      if (indeks == 0) {
44          dodajNaPoczatek(wartosc);
45          return;
46      }
47      if (indeks == rozmiar) {
48          dodajNaKoniec(wartosc);
49          return;
50      }
51
52      Wezel* temp = poczatek;
53      for (int i = 0; i < indeks; ++i) {
54          temp = temp->nastepny;
55      }
56
57      Wezel* nowyWezel = fabryka->utworzWezel(wartosc, temp->
    poprzedni, temp);
58      temp->poprzedni->nastepny = nowyWezel;
59      temp->poprzedni = nowyWezel;
60      rozmiar++;
61  }
62
63  int ListaDwukierunkowa::pobierzRozmiar() const {
64      return rozmiar;
65  }
66
67  void ListaDwukierunkowa::wyswietl() const {
68      Wezel* temp = poczatek;
69      std::cout << "Lista: ";
70      while (temp != nullptr) {
71          std::cout << temp->wartosc << " ";
72          temp = temp->nastepny;
73      }
74      std::cout << std::endl;
```

```
}
```

Listing 8. Plik implementacyjny ListaDwukierunkowa.cpp

4.4. Funkcja główna programu

Plik `main.cpp` integruje poszczególne komponenty i prezentuje testy dodawania elementów.

```cpp
#include <iostream>
#include "fabryka.h"
#include "ListaDwukierunkowa.h"

int main() {
    FabrykaWezlowImpl fabryka;
    ListaDwukierunkowa lista(&fabryka);

    lista.dodajNaPoczatek(10);
    lista.dodajNaPoczatek(5);
    lista.dodajNaKoniec(30);
    lista.dodajPodIndeks(2, 20);

    lista.wyswietl(); // Oczekiwany wynik: 5 10 20 30

    return 0;
}
```

Listing 9. Plik główny main.cpp

5. Wnioski

Bibliografia

[1] Tadeusz Legierski i in. *Programowanie Sterowników PLC*. Gliwice: Wydawnictwo Pracowni Komputerowej Jacka Skalmierskiego, 1998.

[2] *Strona internetowa firmy SELS*. URL: http://www.sels.com.pl/index.php?cPath=1 (dostęp 29.10.2012).

[3] *Strona internetowa TexLive*. URL: https://www.tug.org/texlive/acquire-netinstall.html (dostęp 8.10.2022).

[4] *Strona internetowa TeXstudio*. URL: https://www.texstudio.org (dostęp 8.10.2022).

Spis rysunków

1.1. Logo 4
1.2. Ustawienie TeXstudio 7

Spis tabel

1.1. Tabelka przykładowa 5

Spis listingów

1. Przykładowy kod 001 5
2. Przykładowy kod 002 6
3. Plik nagłówkowy Wezel.h 12
4. Plik implementacyjny Wezel.cpp 12
5. Plik nagłówkowy fabryka.h 12
6. Plik implementacyjny fabryka.cpp 13
7. Plik nagłówkowy ListaDwukierunkowa.h 13
8. Plik implementacyjny ListaDwukierunkowa.cpp 14
9. Plik główny main.cpp 16